CSS183-2 SA01 - Data Extraction and Annotation
Group 19:
NOVERO, Karrin Frida
BALAGAO, Aleah

Frame Extraction Link: https://colab.research.google.com/drive/19P9vNggepQ-K2NSovRhuRtpL8h2cM1zO?usp=drive_link

I. Domain election

A. Selected Domain

This project focuses on the domain of urban traffic object detection. The dataset is composed of real-world traffic intersection videos containing common road objects such as vehicles, motorcycles, buses, and pedestrians. The purpose of selecting this domain is to support computer vision applications related to traffic monitoring, smart transportation systems, and pedestrian safety analysis.

Urban intersections are complex environments where multiple objects interact at the same time. Detecting and identifying these objects is important for improving road safety, traffic management, and automated monitoring systems. Because of this, urban traffic scenes provide a realistic and practical scenario for object detection tasks.

The video sources used in this project were obtained from publicly available YouTube traffic footage and Kaggle traffic-related datasets. These platforms provide accessible real-world recordings captured from different camera angles and lighting conditions.

B. Research Gap

Although many traffic-related datasets are publicly available, several limitations still exist. Some datasets are collected in controlled or simulated environments, which may not fully represent real-world traffic behavior. Others focus only on specific object categories, such as vehicles, without including pedestrians or other road users.

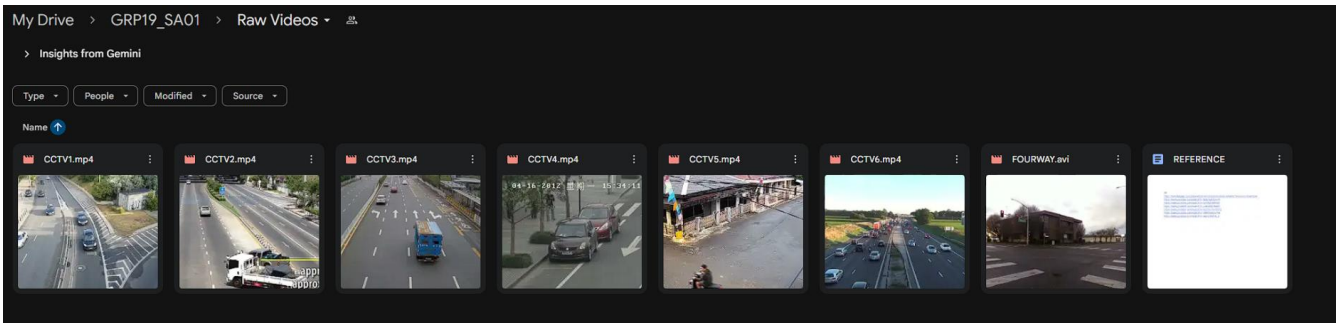
In addition, certain datasets lack diversity in lighting conditions, camera perspectives, and traffic density. This can limit the performance of object detection models when applied to real-world situations.

To address this gap, this project collects real-world intersection footage from different sources, including YouTube and Kaggle, covering various

environments, lighting conditions (daylight, low light), and traffic volumes. Frames were extracted at a controlled sampling rate and annotated using a structured and verified labeling process. This approach helps create a more realistic and diverse dataset suitable for urban traffic object detection tasks.

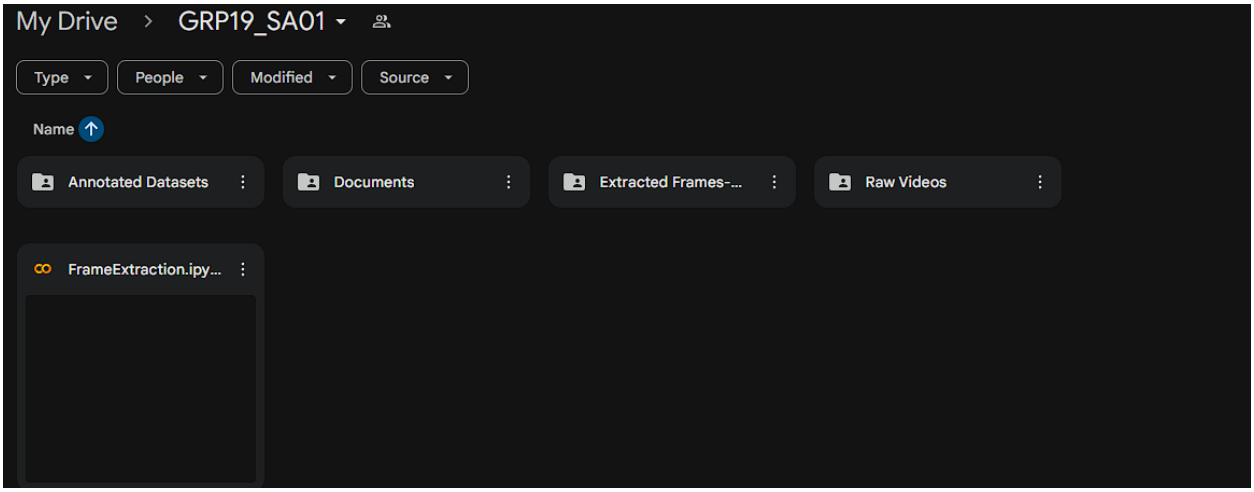
II. Video Collection and Frame Extraction

A. Raw Footage Description



The total colette footage form the jkageel database and Youtube videos are about 16 minutes. The videos were recorded in .mp4 and .avi formats and featured, Urban traffic intersections, Pedestrian crossings, Various vehicle type, Different lighting conditions (daylight, cloudy, low-light scenes), Moderate traffic density

B. Frame Extraction Method



The frame extraction process was performed in Python on Google Colab. Google Drive was mounted to allow easy access to the collected video files and to store the extracted image frames in an organized manner.

The dataset directory was organized as follows:

- Raw Videos – containing original .mp4 and .avi video files
- Extracted Frames-1FPS – containing extracted image frames

- Annotated Datasets – reserved for processed and labeled data

In this section each of the video was processed individually. The program automatically detected video files from the raw video folder and extracted frames at a rate of 1 frame per second (1 FPS). This approach helped reduce repeated images while still capturing meaningful movement in traffic scenes.

```python
import cv2
import os

def extract_frames(video_path, output_folder, fps_rate=1):
    VideoName = os.path.basename(video_path).split('.')[0]
    VideoOutput = os.path.join(output_folder, VideoName)
    os.makedirs(VideoOutput, exist_ok=True)

    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    frameIntervals = int(fps / fps_rate)

    frameCount = 0
    savedCount = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if frameCount % frameIntervals == 0:
            FrameFile = os.path.join(
                VideoOutput,
                f"{VideoName}_frame_{savedCount:05d}.jpg"
            )
            cv2.imwrite(FrameFile, frame)
            savedCount += 1

        frameCount += 1

    cap.release()
    print(f"Finished {VideoName} → Saved {savedCount} frames")
```

The Python function shown above was used to extract frames from each of the video file. The program reads the video using the library OpenCV and retrieves its original frame rate. The code calculates how often a frame should be saved, which we imputed as 1fps. A loop then reads the video frame by frame. At the calculated interval, a frame is saved in .jpg format into a designated folder

corresponding to the video name. The frames are named using a consistent numbering format to maintain proper order.

After processing each video, the program prints the total number of frames saved. This ensures that the extraction process is verified and traceable.

C. Sampling Rate and Dataset Size

```
Videos detected: ['CCTV2.mp4', 'CCTV3.mp4', 'CCTV4.mp4', 'CCTV1.mp4', 'CCTV5.mp4', 'CCTV6.mp4', 'FOURWAY.avi']
Finished CCTV2 → Saved 78 frames
Finished CCTV3 → Saved 48 frames
Finished CCTV4 → Saved 30 frames
Finished CCTV1 → Saved 307 frames
Finished CCTV5 → Saved 314 frames
Finished CCTV6 → Saved 197 frames
Finished FOURWAY → Saved 43 frames
```

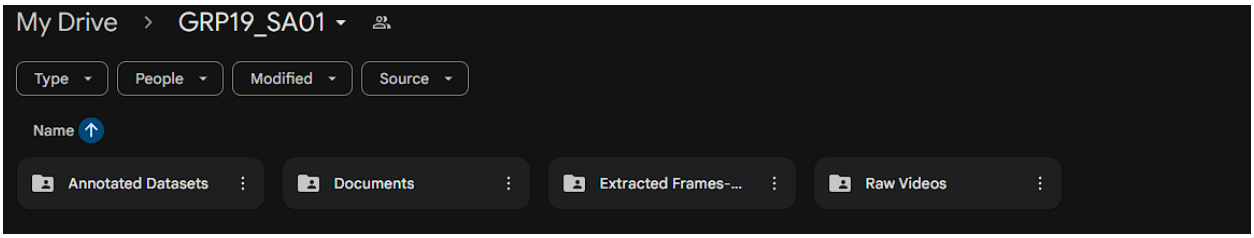
Frames were extracted at a rate of 1 fps. This sampling rate was chosen to avoid saving too many repetitive frames while still capturing meaningful changes in vehicle and pedestrian movement. The extraction process automatically adjusted to each video's original frame rate to ensure consistent sampling across different video formats.

A total of 7 video files were processed, resulting in the following number of extracted frames:

- CCTV1 307
- CCTV2 – 78
- CCTV3 – 48
- CCTV4 – 30
- CCTV5 – 314
- CCTV6 – 197
- FOURWAY – 43

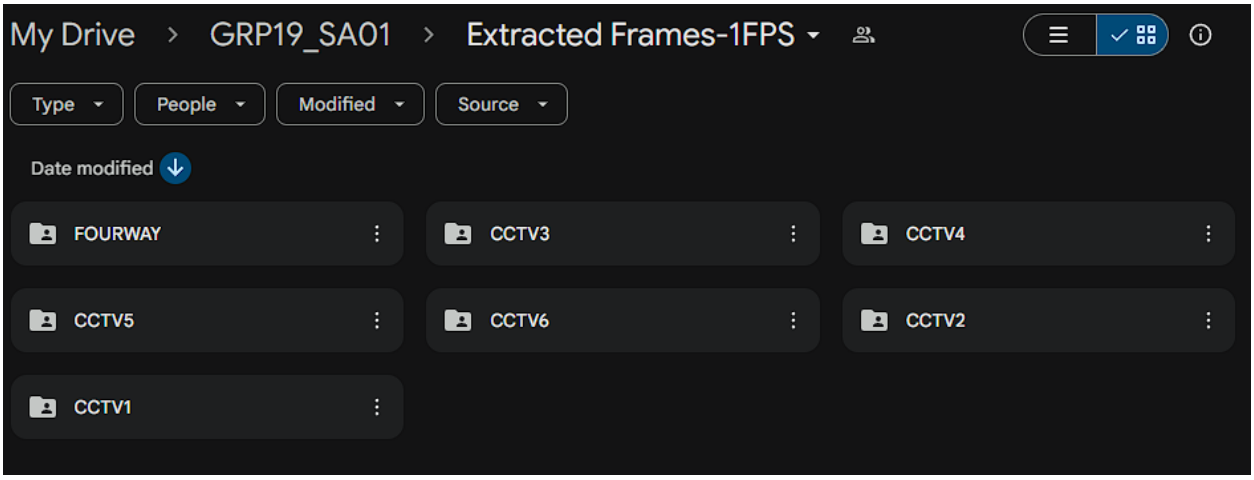
Overall, the extraction process generated about 1000 image frames, which corresponds to approximately 16–17 minutes of raw video footage. This number of frames provides sufficient data diversity for object detection tasks while maintaining a manageable dataset size before annotation.

D. Data Organization Method



To keep the dataset organized and easy to manage, all files were stored systematically in Google Drive. A main project folder named GRP19_SA01 was created to contain all related materials. Inside this folder, subfolders were arranged as follows:

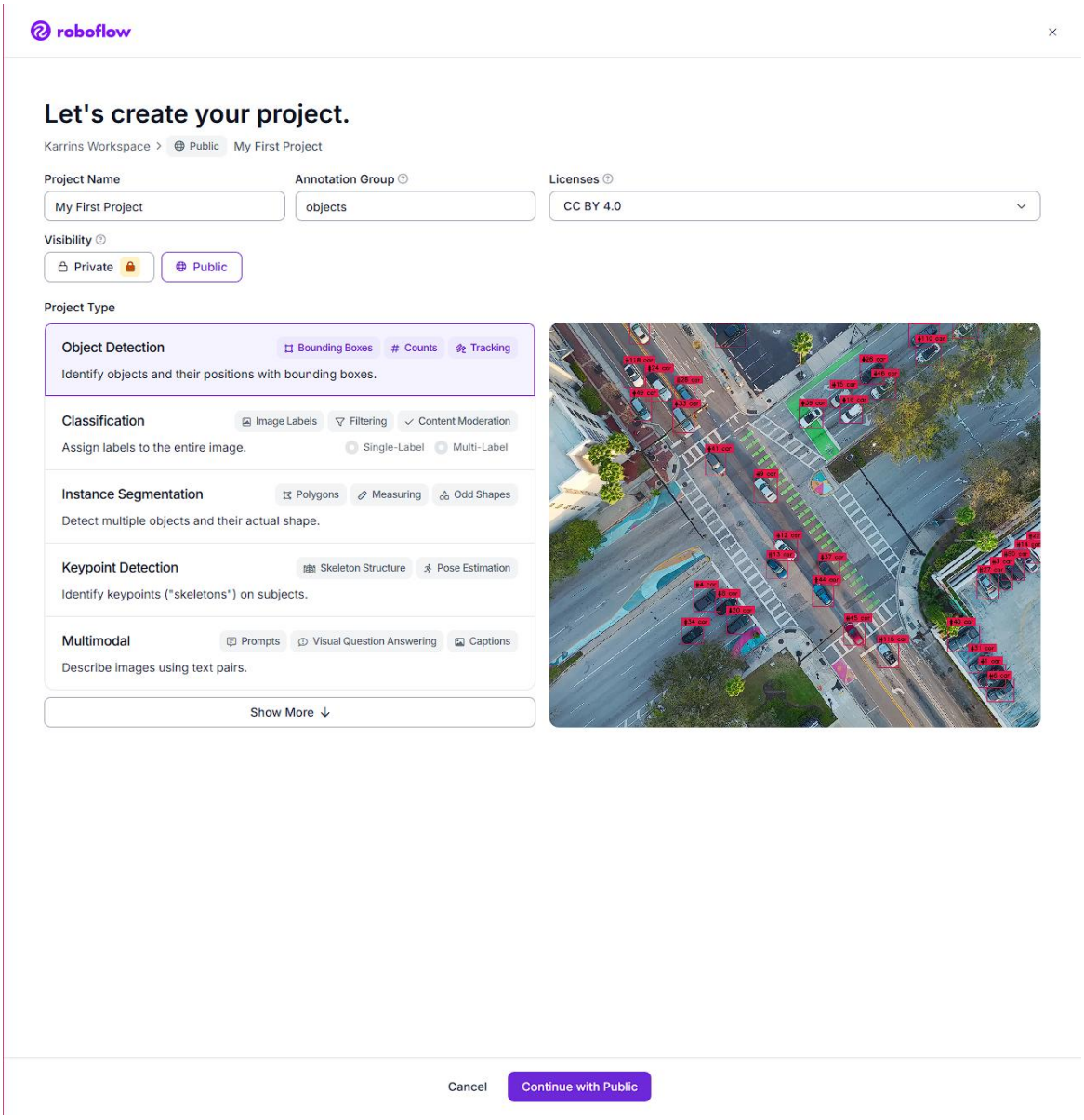
- Raw Videos – contains all original video files used in the project.
- Extracted Frames-1FPS – contains the image frames extracted from each video.
- Annotated Datasets – reserved for labeled data after the annotation stage
- FrameExtraction.ipynb – contains the Python notebook used for frame extraction.



Within the Extracted Frames-1FPS folder, separate subfolders were created for each video sample. This ensures that frames from different videos are not mixed and can be easily traced back to their original video source. Each frame was saved using a consistent naming format to maintain proper order and avoid confusion. This structured organization improves clarity, traceability, and reproducibility of the dataset preparation process before proceeding to annotation.

III. Annotation Process

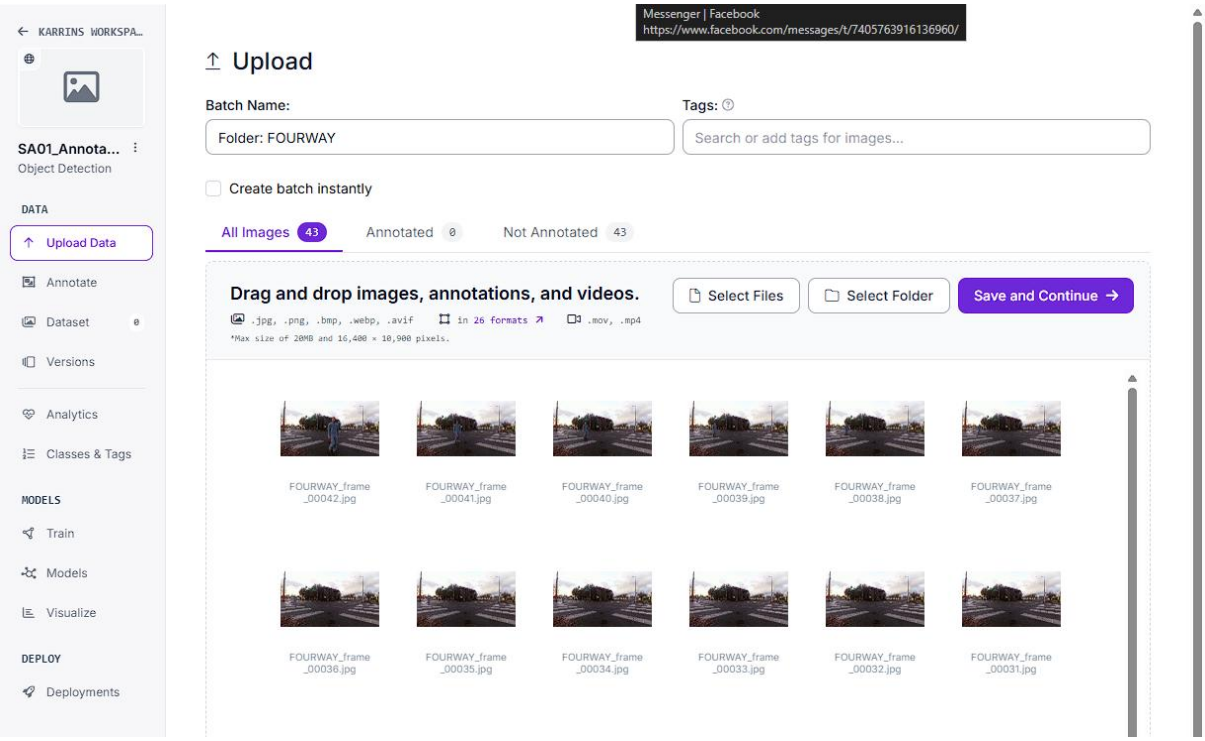
A. Annotation Tool



To start of with the annotation process we used Roboflow, an online dataset management and labeling platform designed for computer vision tasks. Roboflow was selected because it provides automated labeling tools, manual adjustment features, and dataset version control within a single environment.

The tool RoboFlow supports object detection workflows and allows efficient collaboration between automated predictions and human validation. This made it suitable for accelerating the annotation process while still maintaining control over dataset quality.

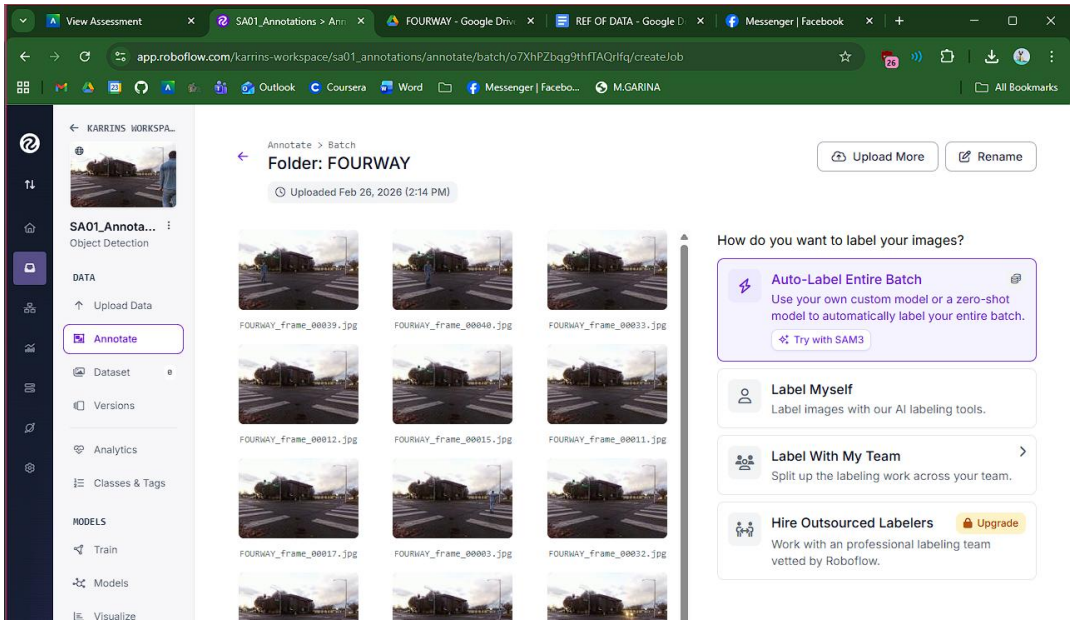
B. Image Importing



All extracted image frames were uploaded into the Roboflow workspace after completing the frame extraction stage. Images were imported in organized batches corresponding to their original video folders (e.g., CCTV1, CCTV2, FOURWAY).

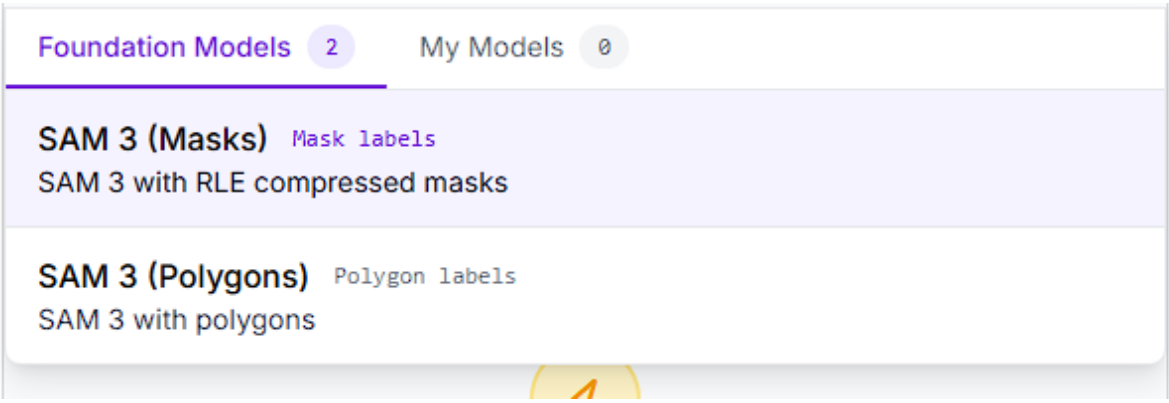
Uploading the images in batches helped preserve traceability between each frame and its source video, allowing easier tracking and verification during the annotation phase.

C. Label and Model Selection Process



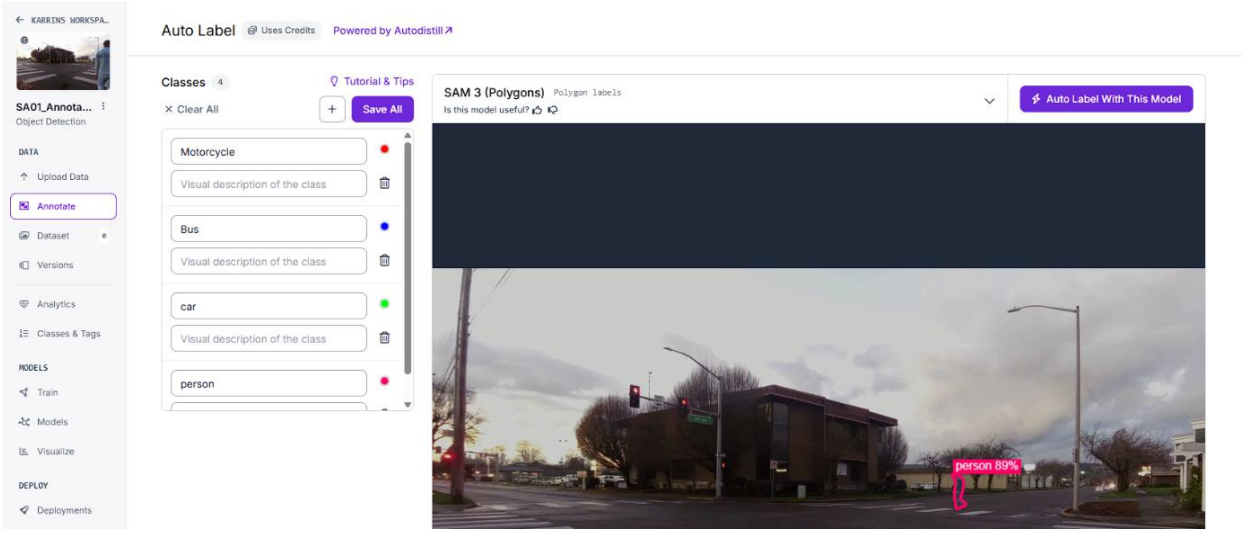
After importing each batch of images, the "Auto-Label Entire Batch" option was selected within the Roboflow annotation interface. This feature allows automated generation of object regions using an AI-assisted segmentation model.

The auto-label option was applied to each uploaded folder separately to maintain organization and ensure that all images in the batch were processed consistently.

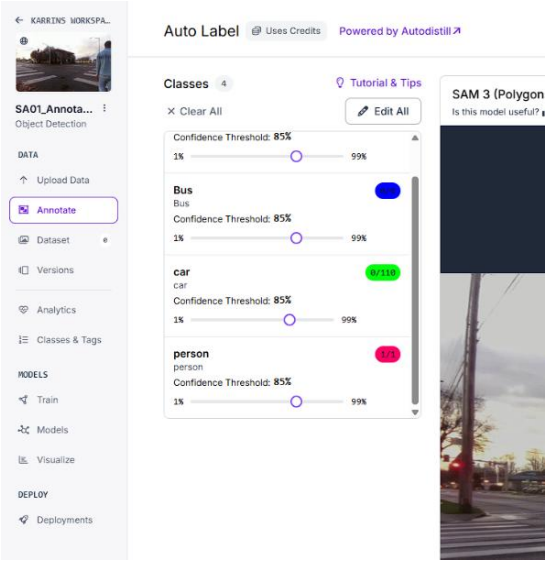


The SAM 3 (Segment Anything Model) was chosen as the labeling model. During this stage, the necessary object classes were added and configured within the platform.

D. Add Class and Adjust Class threshold



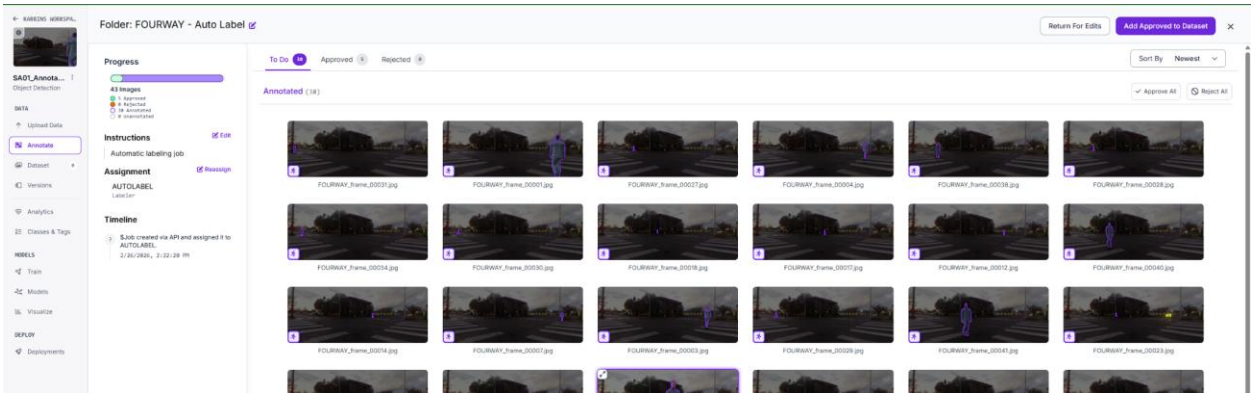
During the auto-label setup, the required object classes were added within the Roboflow interface. Each class was defined according to the types of road users observed in the traffic scenes. These classes were used to categorize the automatically detected objects.



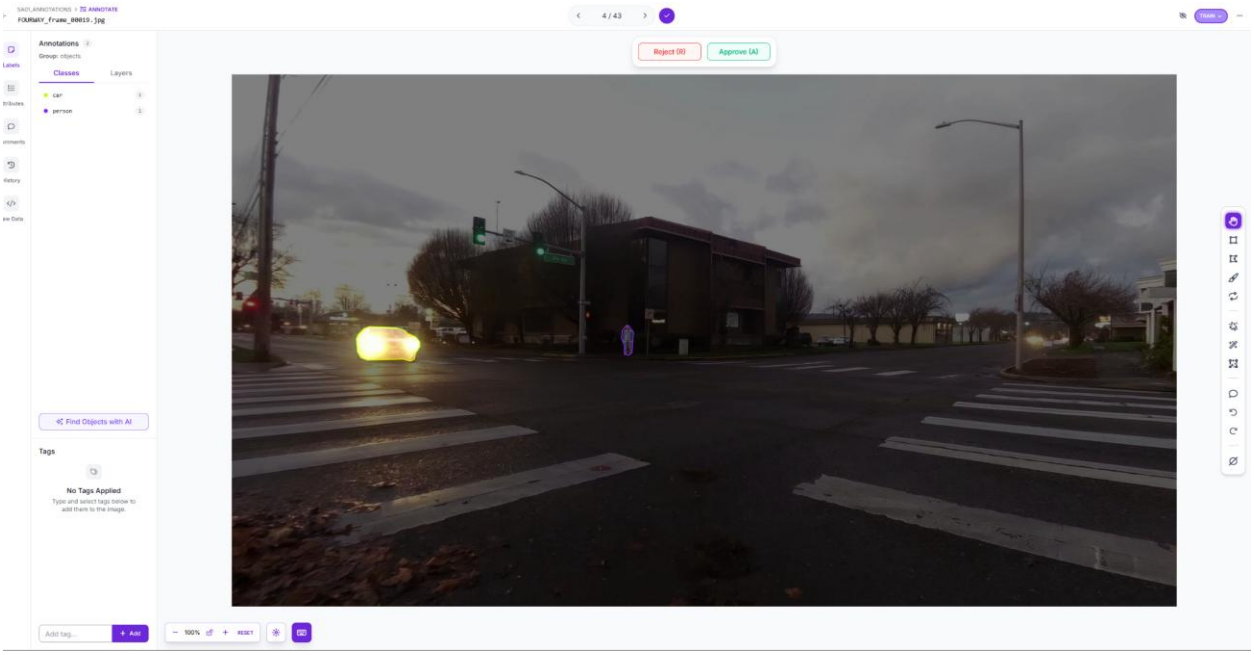
After defining the classes, the confidence threshold for the selected model was adjusted. The threshold was set to 85%, meaning that only detections with at least 85% confidence were generated. This helped reduce incorrect or low-confidence predictions while maintaining sufficient object coverage.

By configuring the class settings and adjusting the confidence threshold before running the model, the auto-labeling process produced more accurate initial annotations, which were later verified during the review stage.

E. Review Annotation



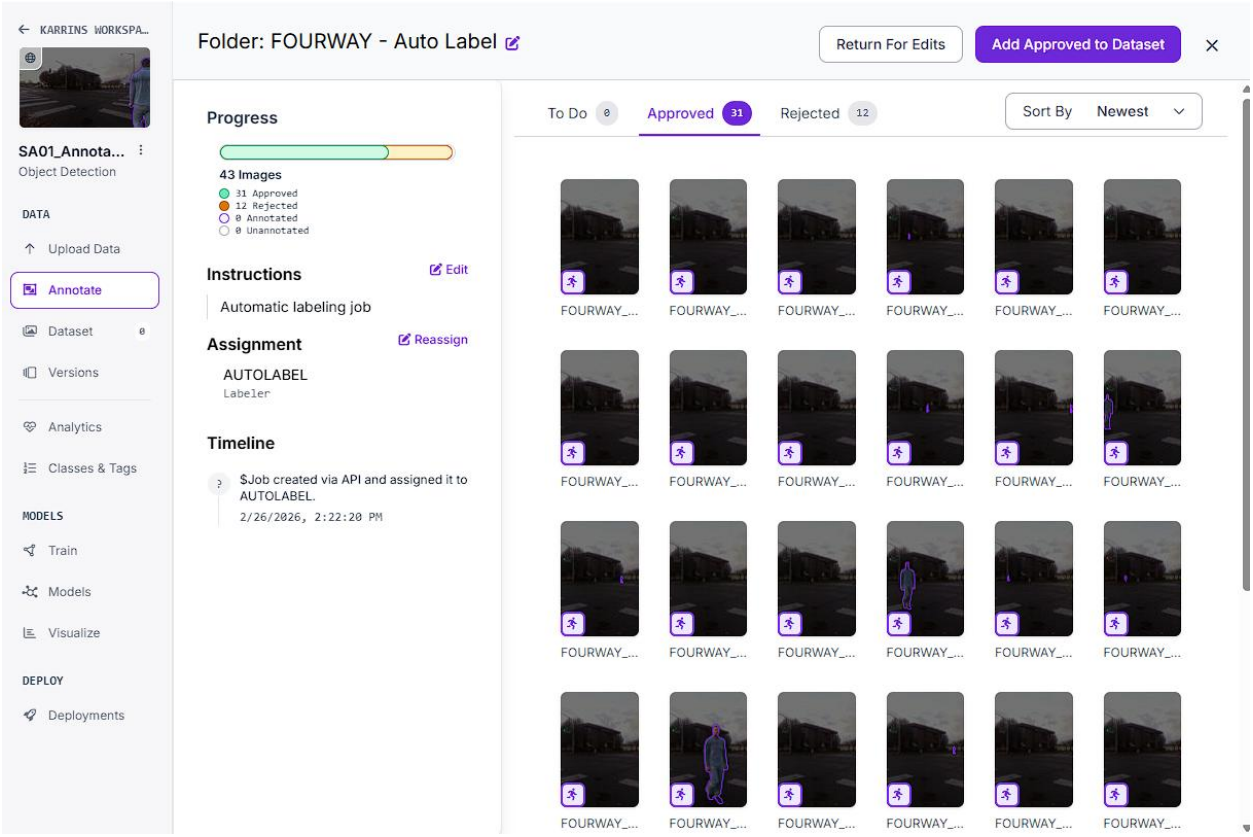
After the auto-labeling process was completed, all images were transferred to the Review section for manual verification. This step is performed to ensure the accuracy and consistency of the automatically generated annotations.



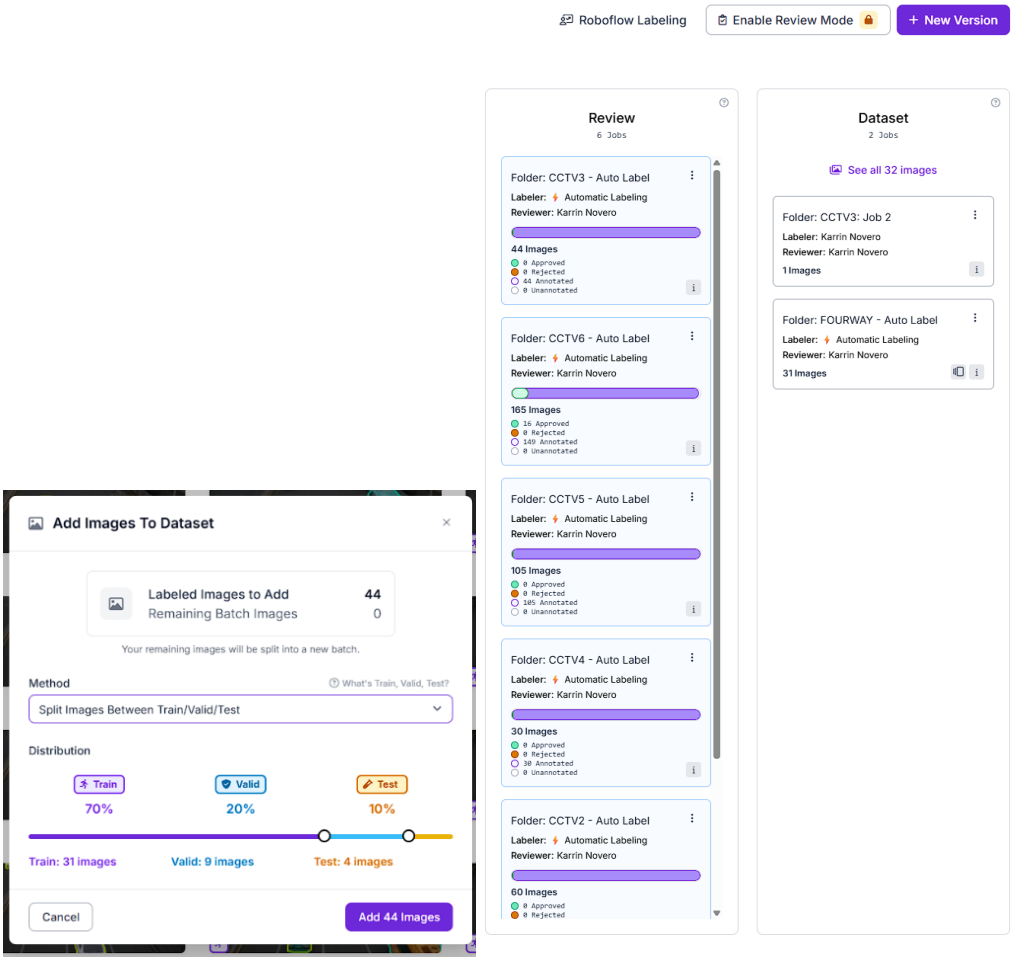
Each image was carefully examined one by one. The generated polygons were checked to confirm that the detected objects were correctly classified and properly outlined. If an annotation was inaccurate, misplaced, or missing, adjustments were made manually using the annotation tools. Incorrect detections were rejected, while valid annotations were accepted

This review step was essential to maintain dataset quality and reduce labeling errors before finalizing the dataset. Only verified and corrected annotations were approved and prepared for inclusion in the final dataset version.

F. Add accepted images to the dataset with the split method



After completing the annotation review, all approved images were added to the main dataset within Roboflow. Only images with verified and corrected annotations were included to ensure data quality.

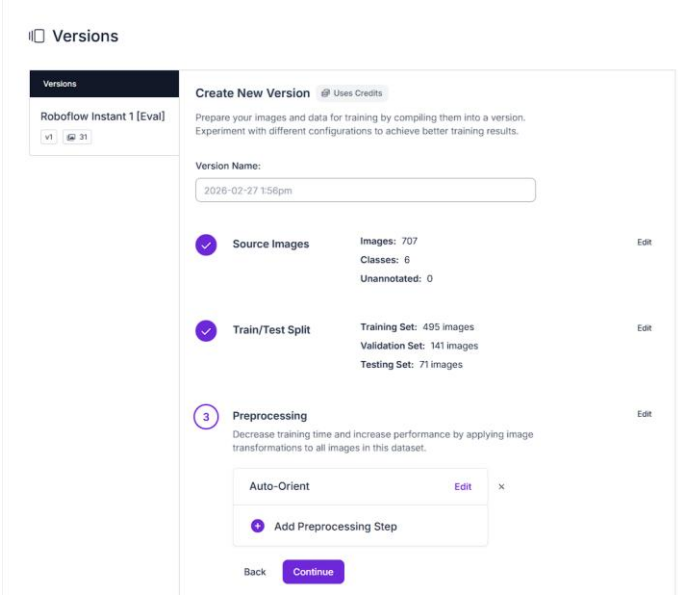


During this stage, the dataset split method was applied. The images were automatically divided into training, validation, and testing sets. This structured division allows the model to learn from one portion of the data (training set), tune performance during development (validation set), and evaluate final performance using unseen data (testing set).

Applying the split method at this stage ensured that the dataset was properly organized for model training while preventing data leakage between sets. The same annotation and review process was then repeated for the remaining video folders until all extracted frames were processed and finalized.

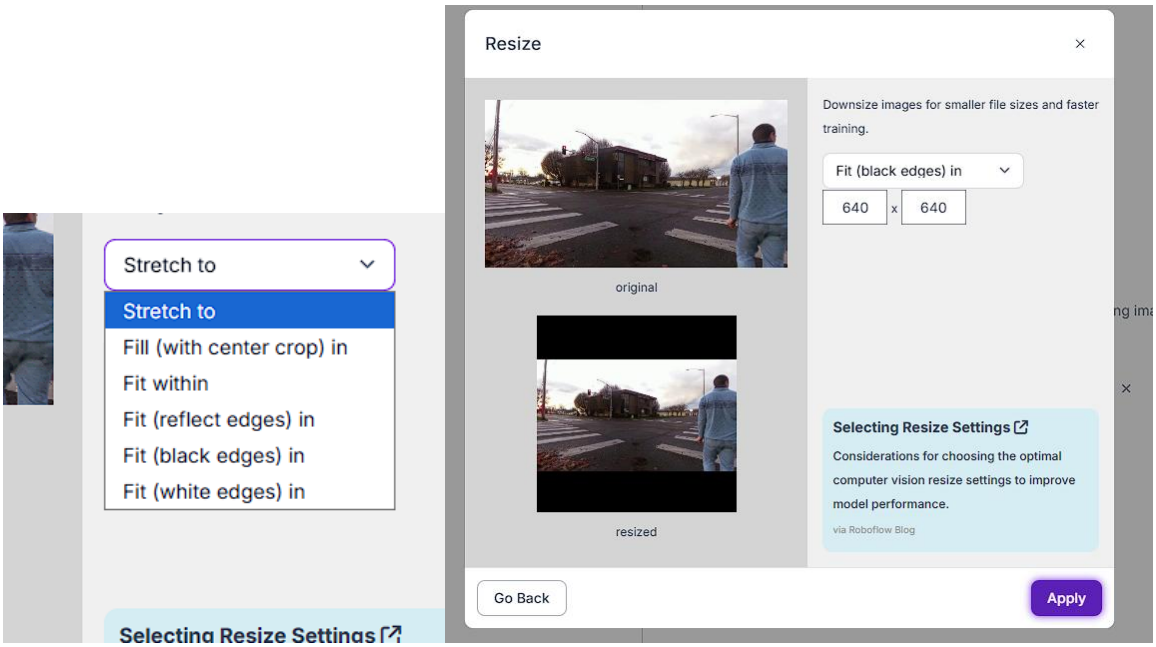
IV. Preprocessing Techniques

A. Auto-Orientation



Before training preparation, the Auto-Orient preprocessing step was applied to all images. This ensured that image orientation metadata was corrected automatically. As a result, all frames were aligned properly regardless of how they were originally recorded or stored. Applying auto-orientation improves dataset consistency and prevents errors caused by rotated or misaligned frames.

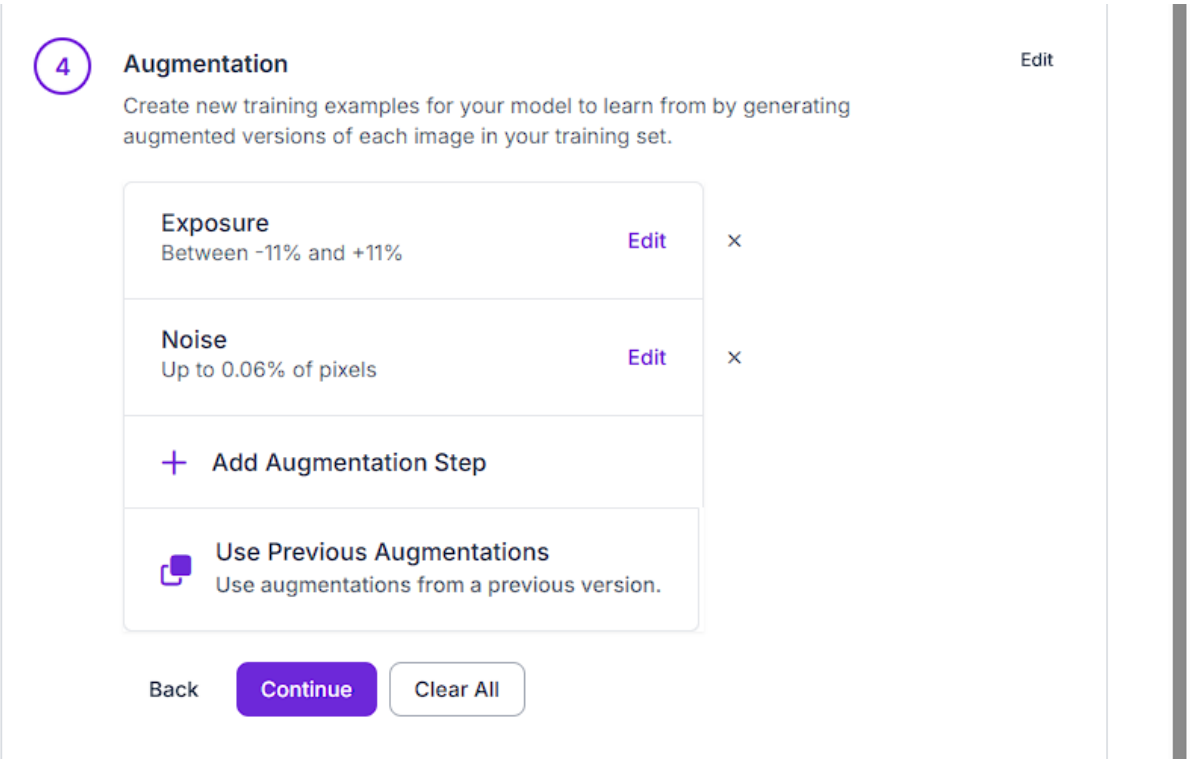
B. Image Resizing



After setting the orientation, All of the images were resized to 640 × 640 pixels using the Fit (black edges) option. With the use of this tool the original

aspect ration of the images will stay and only add black padding when nesseary. Resizing standardizes the input dimensions for model training, ensuring uniformity across the dataset. The selected resolution balances computational efficiency and object detail preservation.

C. Data Augmentation

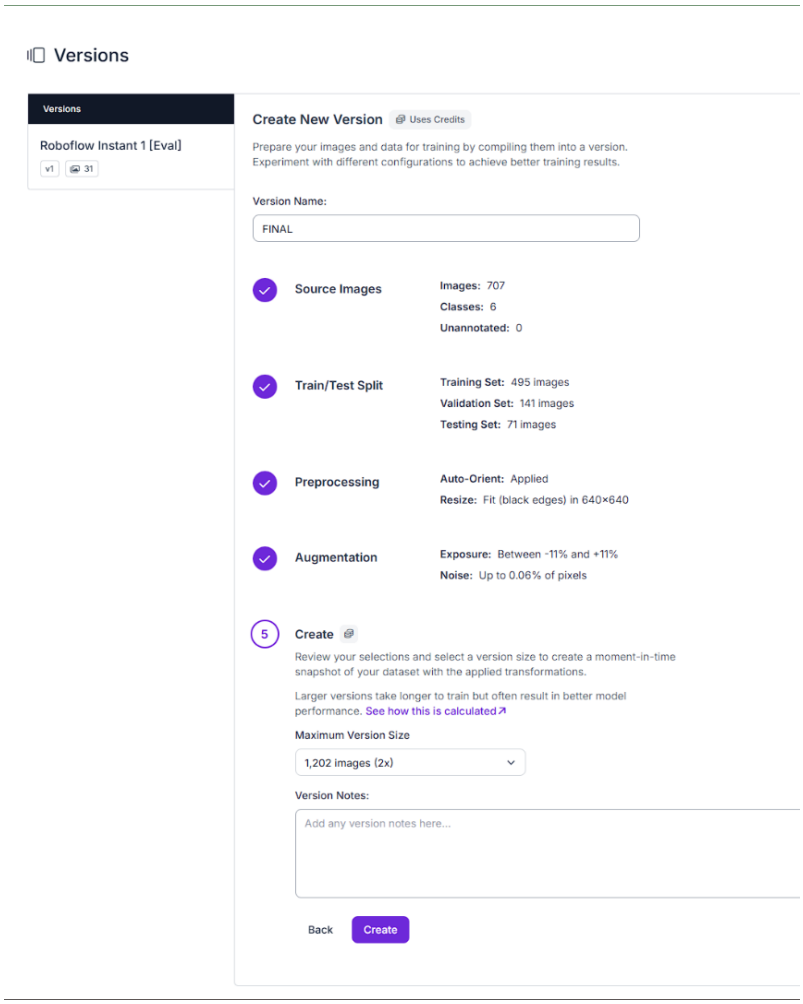


To improve model generalization and simulate real-world variability, selected augmentation techniques were applied to the training set:

- Exposure Adjustment: Between -11% and +11%
- Noise Addition: Up to 0.06% of pixels

Exposure augmentation simulates different lighting conditions such as bright sunlight or dim environments. Noise augmentation introduces slight pixel-level variations to mimic real-world camera imperfections. These augmentations increase dataset diversity while maintaining annotation integrity.

D. Dataset Split Configuration

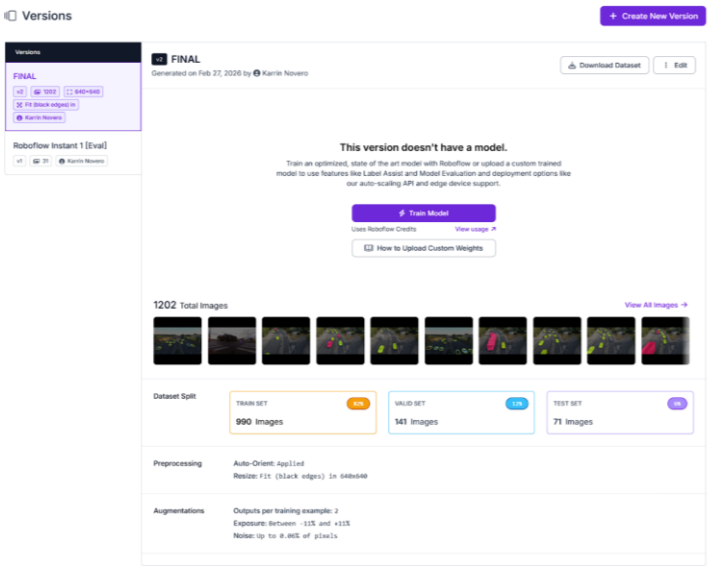


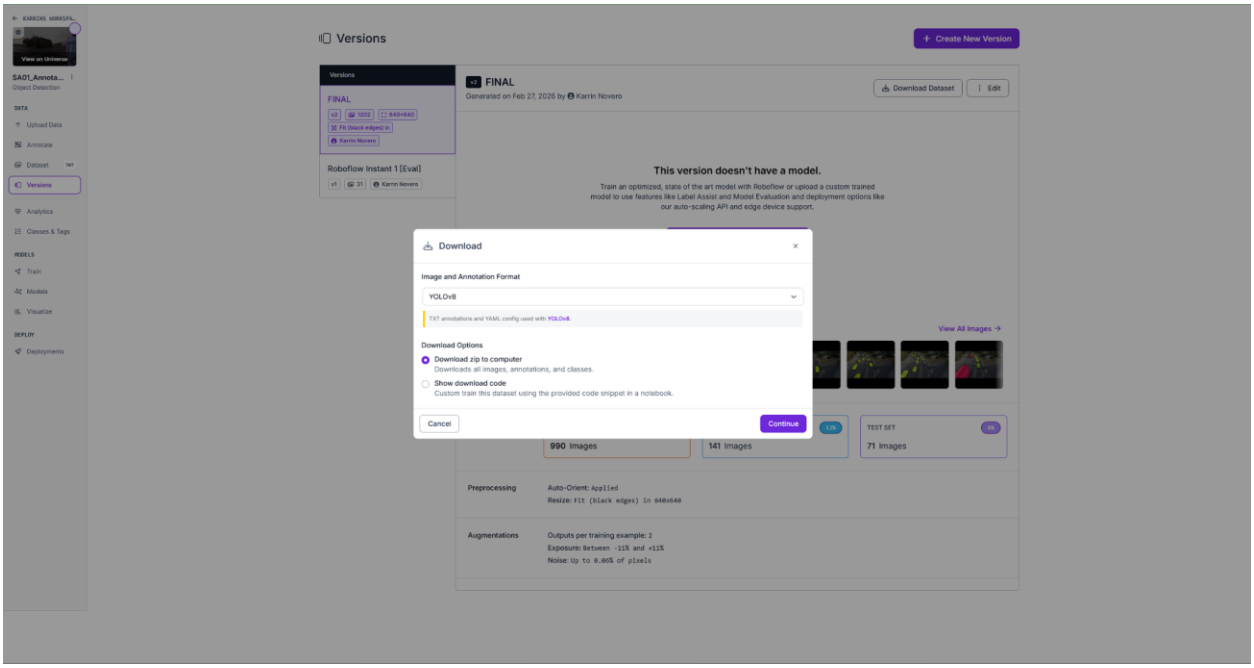
After preprocessing and augmentation configuration of the images, the dataset was divided using the train-validation-test split method:

- Training Set: 495 images
- Validation Set: 141 images
- Testing Set: 71 images

This structured split ensures proper model learning, tuning, and unbiased evaluation.

E. Version Finalization





Once preprocessing and augmentation settings were finalized, a new dataset version was created within Roboflow. This version compiled:

- All annotated images
- Applied preprocessing transformations
- Configured augmentation settings
- Structured dataset split

The finalized version serves as the reproducible dataset snapshot used for future model training and evaluation.

Image normalization was automatically applied during preprocessing. Normalization scales pixel intensity values to a standardized range before model training. This step helps stabilize the learning process by preventing large pixel value variations from negatively affecting model performance. By ensuring consistent pixel distributions across images, normalization improves convergence and overall training reliability.

Reference

https://python.plainenglish.io/step-by-step-using-python-in-google-colab-to-mount-google-drive-and-create-files-and-calculating-18ae9d151323
https://www.geeksforgeeks.org/python/python-program-extract-frames-using-opencv/
https://www.geeksforgeeks.org/python/how-to-load-a-dataset-from-the-google-drive-to-google-colab/
https://www.geeksforgeeks.org/computer-vision/opencv-overview/
https://docs.roboflow.com/annotate/ai-labeling/automated-annotation-with-autodistill
https://www.youtube.com/watch?v=SDV6Gz0suAk
https://www.youtube.com/watch?v=Lh5DhjzZQp0